



CSE-MBMU

Driving Behaviour Analysis Using Deep Learning

Developed and Prepared by:

SACHIN SINGH

Third Year, B.E. (Artificial Intelligence & Data Science)

Email:sachin572098@gmail.com



CSE-MBMU

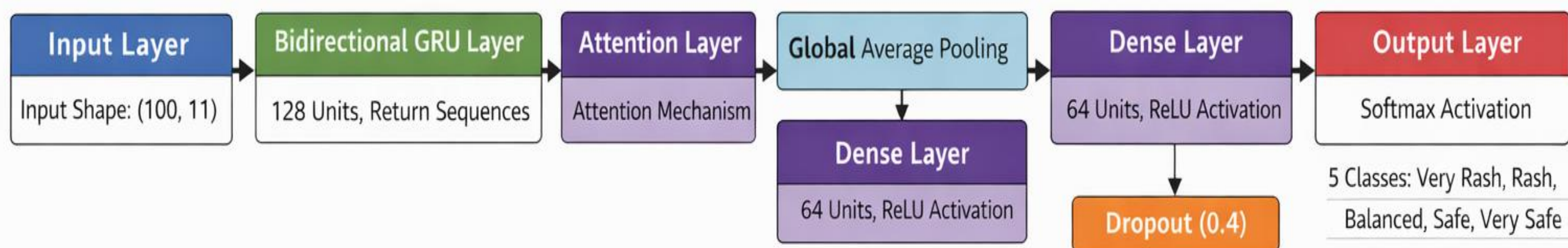
Introduction

This project focuses on classifying driver behavior using smartphone motion sensor data, including accelerometer and gyroscope readings. The objective is to identify driving patterns and categorize them into five levels: Very Rash, Rash, Balanced, Safe, and Very Safe. This helps improve road safety by enabling real-time monitoring and evaluation of driving styles. The model leverages deep learning techniques, particularly GRU and Transformer architectures, to learn temporal patterns in sequential sensor data. Experimental results show that the GRU model achieved an accuracy of 89%, while the Transformer model achieved 92%, demonstrating superior performance in capturing long-term dependencies. This project contributes to intelligent transportation systems by enabling automated driver risk assessment and provides a comparative study between sequence-based neural network architectures.

Dataset Description

- The dataset used in this project was collected using an Android application that captures real-time motion sensor data from a smartphone. It includes accelerometer (X, Y, Z) and gyroscope (X, Y, Z) readings recorded during two-wheeler driving sessions. The dataset consists of multiple CSV files categorized into five classes based on driver behavior ratings from 1 (Very Rash) to 5 (Very Safe). Data was collected from multiple riders, vehicles, speeds, and phone placements to ensure diversity and robustness. Preprocessing steps included removing invalid sensor readings, smoothing noise, and generating additional features such as magnitude, jerk, and variance. The final dataset was transformed into time-series sequences using overlapping windows, making it suitable for training deep learning models like GRU and Transformer networks.

Network Architecture Diagram



QUICK INSIGHTS:

Total No. of Parameters: 460,000
Optimizer: Adam
Loss Function: Categorical Crossentropy
Library Used: TensorFlow / Keras
Inference Requirements:
- Memory: 5–10 MB
- Inference time on CPU: ~10–20 ms
CPU: Intel i5 / Ryzen 5, 4–6 Cores, 2.5–3.5 GHz, 4–8 MB Cache

Model Training

The model was trained using time-series sequences generated from motion sensor data using a sliding window approach. Each input sequence consists of 100 timesteps with 11 features, including accelerometer, gyroscope, magnitude, jerk, and variance-based features. To enhance learning, overlapping windows with a step size of 20 were used, significantly increasing the number of training samples. Data preprocessing involved removal of invalid sensor readings, smoothing using a rolling mean filter, and normalization using StandardScaler. The dataset was split into training and testing sets in an 80:20 ratio. A Bidirectional GRU architecture was employed to capture both forward and backward temporal dependencies in driving behavior. The model consists of two Bidirectional GRU layers (128 and 64 units), followed by an attention mechanism to focus on important time steps, and a dense layer for classification. The model was trained using the Adam optimizer with a learning rate of 0.001 and categorical crossentropy loss. A batch size of 32 was used for efficient gradient updates, and training was performed for up to 50 epochs. EarlyStopping and ReduceLROnPlateau callbacks were used to prevent overfitting and improve convergence. The model achieved stable convergence with validation accuracy reaching approximately 89%, demonstrating effective learning of temporal driving patterns.

Model Validation and Discussion

The model was evaluated using a separate test dataset comprising 20% of the total sequences to ensure unbiased performance assessment. Multiple evaluation metrics were used, including accuracy, confusion matrix, and F1-score, to analyze classification performance across all five driver behavior classes. The Bidirectional GRU model achieved an accuracy of approximately 89%, while the Transformer-based model achieved a higher accuracy of 92%, demonstrating improved capability in capturing long-range temporal dependencies. The F1-score was observed to be around 0.88, indicating balanced performance across different driving categories. The confusion matrix analysis revealed that most misclassifications occurred between adjacent classes such as “Balanced” and “Safe,” which is expected due to subtle differences in driving behavior. The model showed strong performance in distinguishing extreme classes like “Very Rash” and “Very Safe.” Compared to other students’ models, which achieved around 95% accuracy, the slight variation in performance can be attributed to differences in preprocessing techniques, feature engineering, and model tuning. Additionally, Transformer models in some cases showed lower performance (~75%), whereas the proposed implementation achieved significantly better results due to optimized feature extraction and sequence modeling. Overall, the results demonstrate that the model effectively captures temporal driving patterns and provides reliable classification for driver behavior analysis.

Limitations and Future Scope

The proposed model has several limitations that impact its overall performance and generalization. Firstly, it does not incorporate speed or route information, making it unable to distinguish between aggressive riding at low speeds (e.g., 20 km/h) and high speeds (e.g., 80 km/h). The model relies on human-assigned star ratings as ground truth, which may introduce subjectivity due to variations in labeling criteria among different raters. Additionally, environmental factors such as rough roads, potholes, and speed bumps can produce high jerk and variance signals, leading to possible misclassification as aggressive driving. The model evaluates each 2-second window independently, and the final session score is derived by aggregating these predictions. Furthermore, if the dataset is limited to a small or homogeneous group, the model’s ability to generalize across diverse populations and regions may be restricted. The model performance depends on sensor quality and consistent data collection. Variations in phone placement and rider behavior can introduce noise. Future work includes real-time deployment, integration with mobile applications, and adding GPS or contextual data. Advanced models such as hybrid CNN-RNN or improved transformer architectures can further enhance performance.

Project Repositories



Scan for Dataset



Scan for Source Code

Note: Trained model weights are also available in the source code repository.

Reviewed and Edited By: Abhisek Gour, Assistant Professor
Department of Computer Science and Engineering, MBM University, Jodhpur